

# Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



## 2028\_NeoColab\_Design and Analysis of AlgorithmsS1L3

### VTU\_DAA\_Week 10\_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 48.5

### Section 1 : Coding

#### 1. Problem Statement

Arun, the operations manager of a logistics company, needs to assign three delivery drivers to three delivery zones.

Each driver requires a different amount of fuel (in liters) to complete deliveries in each zone due to varying distances and traffic conditions. Arun wants to assign exactly one driver to each zone such that the total fuel consumption is minimized.

To efficiently explore possible assignments and avoid unnecessary computations, Arun decides to use the Branch and Bound technique.

Your task is to help Arun determine the optimal assignment of drivers to zones that results in the least total fuel consumption.

### ***Input Format***

The input consists of three lines with three space-separated integers each, representing a  $3 \times 3$  matrix.

Each cell (i, j) represents the fuel required (in liters) if the ith driver is assigned to the jth zone.

### ***Output Format***

The output should display the optimal assignment in the following format:

Optimal Assignment:

Driver 1 -> Zone X

Driver 2 -> Zone Y

Driver 3 -> Zone Z

Where:

Each driver is assigned to exactly one zone.

Each zone is assigned to exactly one driver.

The assignment must produce the least total fuel consumption.

Refer to the sample output for formatting specifications.

### ***Sample Test Case***

Input: 8 6 10

9 7 4

3 12 5

Output: Optimal Assignment:

Driver 1 -> Zone 2

Driver 2 -> Zone 3

Driver 3 -> Zone 1

### ***Answer***

```
#include <stdio.h>
#include <limits.h>
```

```
int cost[3][3];
int best[3], temp[3];
int used[3];
int minCost = INT_MAX;
```

```
void solve(int i, int curr) {
    if (i == 3) {
        if (curr < minCost) {
            minCost = curr;
            for (int k = 0; k < 3; k++)
                best[k] = temp[k];
        }
        return;
    }
```

```
    if (curr >= minCost)
        return;
```

```
    for (int j = 0; j < 3; j++) {
        if (!used[j]) {
            used[j] = 1;
            temp[i] = j;
            solve(i + 1, curr + cost[i][j]);
            used[j] = 0;
        }
    }
}
```

```
int main() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            scanf("%d", &cost[i][j]);
```

```
    solve(0, 0);
```

```
    printf("Optimal Assignment:\n");
    for (int i = 0; i < 3; i++)
        printf("Driver %d -> Zone %d\n", i + 1, best[i] + 1);
```

```
    return 0;  
}
```

**Status :** Correct

**Marks :** 10/10

## 2. Problem Statement

Captain Arvind has discovered a hidden island filled with valuable treasures. Before leaving the island, he must load the treasures onto his small ship. However, the ship has a strict weight capacity, so he cannot carry everything he finds.

Each treasure chest has a specific value and weight. Captain Arvind wants to select the best combination of treasure chests to load onto the ship so that the total value of the treasures is maximized without exceeding the ship's weight limit.

To solve this efficiently, Captain Arvind decides to use the Branch and Bound technique applied to the 0/1 Knapsack problem, which systematically explores possible combinations and eliminates choices that cannot produce better results.

Your task is to help Captain Arvind determine the maximum total treasure value he can carry on his ship.

### **Input Format**

The first line contains an integer  $n$ , representing the total number of treasure chests.

The next  $n$  lines contain two space-separated integers:  $val[i]$   $w[i]$

where

- $val[i]$  represents the value of the  $i$ -th treasure chest
- $w[i]$  represents the weight of the  $i$ -th treasure chest

The last line contains an integer  $W$ , representing the maximum weight capacity of the ship.

### **Output Format**

The output prints "Optimal Total Value: X" , where X is the maximum value that Captain Arvind can obtain using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 3

60 10

100 20

120 30

50

Output: Optimal Total Value: 220

### **Answer**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int val, wt;
```

```
    double ratio;
```

```
} Item;
```

```
int cmp(const void *a, const void *b) {
```

```
    Item *i1 = (Item *)a;
```

```
    Item *i2 = (Item *)b;
```

```
    if (i1->ratio < i2->ratio) return 1;
```

```
    if (i1->ratio > i2->ratio) return -1;
```

```
    return 0;
```

```
}
```

```
double bound(int n, int W, int idx, int wt, int val, Item items[]) {
```

```
    double profit = val;
```

```
    int w = wt;
```

```
    for (int i = idx; i < n; i++) {
```

```
        if (w + items[i].wt <= W) {
```

```

        w += items[i].wt;
        profit += items[i].val;
    } else {
        profit += (W - w) * items[i].ratio;
        break;
    }
}
return profit;
}

```

```
int maxProfit = 0;
```

```

void knapsack(int n, int W, int idx, int wt, int val, Item items[]) {
    if (wt <= W && val > maxProfit)
        maxProfit = val;

    if (idx == n)
        return;

    if (bound(n, W, idx, wt, val, items) <= maxProfit)
        return;

    if (wt + items[idx].wt <= W)
        knapsack(n, W, idx + 1, wt + items[idx].wt, val + items[idx].val, items);

    knapsack(n, W, idx + 1, wt, val, items);
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    Item items[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].val, &items[i].wt);
        items[i].ratio = (double)items[i].val / items[i].wt;
    }

    int W;
    scanf("%d", &W);

    qsort(items, n, sizeof(Item), cmp);
}

```

```
knapsack(n, W, 0, 0, 0, items);  
  
printf("Optimal Total Value: %d", maxProfit);  
  
return 0;  
}
```

**Status :** Correct

**Marks :** 10/10

### 3. Problem Statement

Dr. Aarav is a field scientist preparing for an important research expedition to a remote mountain region. Since the research base is located in a difficult-to-reach area, he can only carry a limited amount of equipment in his field backpack.

Each piece of equipment has a scientific importance value and a specific weight. Dr. Aarav wants to choose the best combination of equipment so that the total scientific value is maximized without exceeding the backpack's weight capacity.

To solve this efficiently, Dr. Aarav applies the Branch and Bound technique using the 0/1 Knapsack algorithm, which systematically explores possible selections and eliminates combinations that cannot yield better results.

Your task is to help Dr. Aarav determine which equipment weights should be selected to achieve the maximum total value.

**Note:** The weights must be printed in the same order as they appear in the input.

#### ***Input Format***

The first line of input contains an integer  $n$ , representing the number of available equipment items.

The second line contains  $n$  space-separated integers  $val[i]$ , representing the scientific value of each item.

The third line contains n space-separated integers  $w[i]$ , representing the weight of each item.

The fourth line contains an integer  $W$ , representing the maximum carrying capacity of the backpack.

### **Output Format**

The output displays the "Specified weights: <weights>", where <weights> are the weights of the selected equipment items that maximize the total value using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample outputs for the exact format.

### **Sample Test Case**

Input: 4

10 20 30 40

1 2 3 4

8

Output: Specified weights: 1 3 4

### **Answer**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int val, wt, idx;  
    double ratio;  
} Item;
```

```
int n, W;
```

```
Item items[10];
```

```
int bestSet[10], currSet[10];
```

```
int maxProfit = 0;
```

```
int cmp(const void *a, const void *b) {
```

```
    Item *i1 = (Item *)a;
```

```
    Item *i2 = (Item *)b;
```

```
    if (i1->ratio < i2->ratio) return 1;
```

```
    if (i1->ratio > i2->ratio) return -1;
```



```

    return 0;
}

double bound(int idx, int wt, int val) {
    double profit = val;
    int w = wt;

    for (int i = idx; i < n; i++) {
        if (w + items[i].wt <= W) {
            w += items[i].wt;
            profit += items[i].val;
        } else {
            profit += (W - w) * items[i].ratio;
            break;
        }
    }
    return profit;
}

```

```

void knapsack(int idx, int wt, int val) {
    if (wt <= W && val > maxProfit) {
        maxProfit = val;
        for (int i = 0; i < n; i++)
            bestSet[i] = currSet[i];
    }
}

```

```

if (idx == n)
    return;

```

```

if (bound(idx, wt, val) <= maxProfit)
    return;

```

```

if (wt + items[idx].wt <= W) {
    currSet[items[idx].idx] = 1;
    knapsack(idx + 1, wt + items[idx].wt, val + items[idx].val);
    currSet[items[idx].idx] = 0;
}

```

```

    knapsack(idx + 1, wt, val);
}

```

```

int main() {

```

```

scanf("%d", &n);

int val[10], wt[10];
for (int i = 0; i < n; i++)
    scanf("%d", &val[i]);
for (int i = 0; i < n; i++)
    scanf("%d", &wt[i]);

scanf("%d", &W);

for (int i = 0; i < n; i++) {
    items[i].val = val[i];
    items[i].wt = wt[i];
    items[i].idx = i;
    items[i].ratio = (double)val[i] / wt[i];
    currSet[i] = 0;
    bestSet[i] = 0;
}

qsort(items, n, sizeof(Item), cmp);

knapsack(0, 0, 0);

printf("Specified weights:");
for (int i = 0; i < n; i++) {
    if (bestSet[i])
        printf(" %d", wt[i]);
}

return 0;
}

```

**Status :** Correct

**Marks :** 10/10

#### 4. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

### ***Input Format***

The first line of input consists of an integer  $n$ , representing the number of cities.

The next  $n$  lines contain  $n$  space-separated integers each, representing the distance matrix between the cities.

The cell  $(i, j)$  represents the distance from city  $i$  to city  $j$ .

### ***Output Format***

The first line of output displays "The Path is: " followed by the path of the route using  $\text{--->}$  in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

### ***Sample Test Case***

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1 $\text{--->}$ 3 $\text{--->}$ 2 $\text{--->}$ 4 $\text{--->}$ 1

Minimum cost is 7

### ***Answer***

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;  
int cost[10][10];  
int visited[10];  
int bestPath[11], currPath[11];  
int minCost = INT_MAX;  
  
void tsp(int curr, int count, int currCost) {  
    if (count == n && cost[curr][0] > 0) {  
        int total = currCost + cost[curr][0];  
        if (total < minCost) {  
            minCost = total;  
            for (int i = 0; i < n; i++)  
                bestPath[i] = currPath[i];  
            bestPath[n] = 0;  
        }  
        return;  
    }  
}
```

```
for (int i = 0; i < n; i++) {  
    if (!visited[i] && cost[curr][i] > 0) {  
        int temp = currCost + cost[curr][i];  
        if (temp >= minCost)  
            continue;  
  
        visited[i] = 1;  
        currPath[count] = i;  
        tsp(i, count + 1, temp);  
        visited[i] = 0;  
    }  
}
```

```
int main() {  
    scanf("%d", &n);  
  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < n; j++)  
            scanf("%d", &cost[i][j]);  
  
    for (int i = 0; i < n; i++)  
        visited[i] = 0;
```

```

visited[0] = 1;
currPath[0] = 0;

tsp(0, 1, 0);

printf("The Path is: ");
for (int i = 0; i <= n; i++) {
    printf("%d", bestPath[i] + 1);
    if (i < n)
        printf("--->");
}

printf("\nMinimum cost is %d", minCost);

return 0;
}

```

**Status :** Partially correct

**Marks :** 8.5/10

## 5. Problem Statement

A traveling salesman is tasked with finding the shortest route to visit a set of cities and return to the starting city. The goal is to minimize the total distance traveled by the salesman while visiting each city exactly once.

### **Input Format**

The first line contains an integer  $n$ , the number of cities.

The next  $n$  lines contain  $n$  space-separated integers each, representing the distances between cities.

The  $i$ th line represents the distances from city  $i$  to all other cities.

The last line contains an integer  $s$ , representing the starting city.

### **Output Format**

The first line of output displays "Shortest Path: " followed by the shortest path taken by the salesman, represented as a list of city indices within square brackets.

The second line of output displays "Minimum Cost: " followed by the minimum cost (total distance) of the shortest path.

Refer to the sample output for the formatting specifications.

### **Sample Test Case**

Input: 4  
0 10 15 20  
10 0 35 25  
15 35 0 30  
20 25 30 0  
0

Output: Shortest Path: [0, 1, 3, 2, 0]  
Minimum Cost: 80

### **Answer**

```
#include <stdio.h>
#include <limits.h>
```

```
int n, s;
int cost[10][10];
int visited[10];
int currPath[11], bestPath[11];
int minCost = INT_MAX;
```

```
void tsp(int curr, int count, int currCost) {
    if (count == n && cost[curr][s] > 0) {
        int total = currCost + cost[curr][s];
        if (total < minCost) {
            minCost = total;
            for (int i = 0; i < n; i++)
                bestPath[i] = currPath[i];
            bestPath[n] = s;
        }
        return;
    }
}
```

```
for (int i = 0; i < n; i++) {
    if (!visited[i] && cost[curr][i] > 0) {
```

```

        int temp = currCost + cost[curr][i];
        if (temp >= minCost)
            continue;

        visited[i] = 1;
        currPath[count] = i;
        tsp(i, count + 1, temp);
        visited[i] = 0;
    }
}
}

```

```

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    scanf("%d", &s);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    visited[s] = 1;
    currPath[0] = s;

    tsp(s, 1, 0);

    printf("Shortest Path: [");
    for (int i = 0; i <= n; i++) {
        printf("%d", bestPath[i]);
        if (i < n)
            printf(", ");
    }
    printf("]\n");

    printf("Minimum Cost: %d", minCost);

    return 0;
}

```

**Status :** Correct

**Marks :** 10/10